Estruturas de Dados Básicas II

Prof. Dr. Matheus da Silva Menezes



UFRN
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

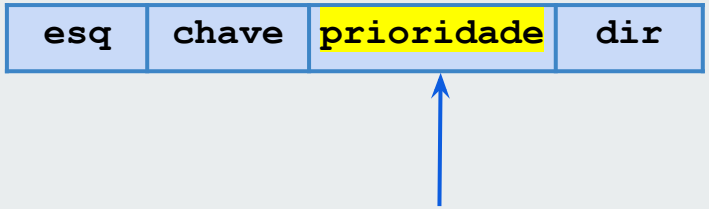
Unidade 02.

Estruturas de Dados Básicas II

HEAPSORT
Construção da Heap e Ordenação



Uma representação para nó de uma árvore binária / heap



Exemplo C

```
struct nodo{
  int chave;
  int prioridade;
  struct nodo *esq;
  struct nodo *dir; };
typedef struct nodo no;
}
```

```
typedef no *pont_no;
```

Observação: O campo chave será usado como chave e conteúdo/prioridade

Definição

- **Uma lista de prioridade é uma tabela na qual a cada um de seus dados está associada uma prioridade.**
- **Uma estrutura de dados heap é uma lista linear que pode ser visualizada como uma árvore binária completa, com o último nível preenchido da esquerda para a direita.**



Definição

Propriedade de Heap:

- *Em um Heap, todo nó deve ter prioridade maior (menor) do que a dos seus filhos (caso existam)*

MAX

$s_i \leq s_{\lfloor i/2 \rfloor}$ para $1 \leq i \leq n$

16	14	10	8	7	9	3	2	4	1



H é o vetor que armazena a heap.

H tem comprimento **l** e a heap tem comprimento h_s, $h_s \leq l$.

A raiz da árvore é **H[1]**. Dado **i** o índice de um nó, os índices do seu pai e filhos são dados por:

- `Pai(i) = i div 2`
- `Filho_esq(i) = 2i`
- `Filho_dir(i) = 2i + 1`



Operações Básicas

1. ALTERAÇÃO DE PRIORIDADE
2. INSERÇÃO
3. REMOÇÃO
4. CONSTRUÇÃO

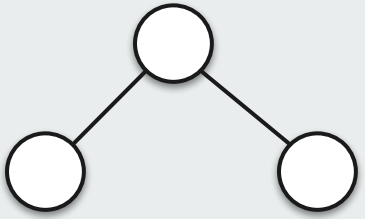
CONSTRUÇÃO

Para a execução de operações básicas de heaps, já foram desenvolvidas várias funções auxiliares

A idéia geral é aproveitar as estruturas e funções já desenvolvidas para construir a heap.

Pseudocódigo

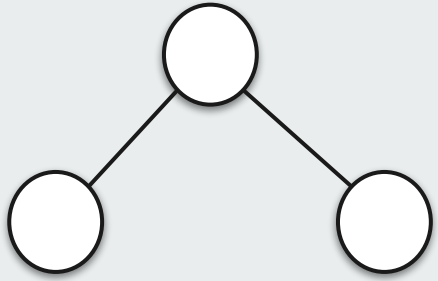
```
descer(lista H, int i){
    maior = i;
    se (2*i<= H.tamanho e H[2*i]< H[maior]))
        maior = 2*i; //Se o filho da esquerda é maior, atualiza
    se (2*i+1<= H.tamanho e H[2*i+1]< H[maior]))
        maior = 2*i+1; //Se o filho da direita é maior, atualiza
    se (maior != i){
        troca(H[i],H[maior]); //troca raiz com maior dos filhos
       descer(H, maior);
    }
}
```





Pseudocódigo

```
subir(lista H, int i){
    pai = ⌊i/2⌋;
    se (i >1 e H[i]> H[pai])){
      troca(H[i],H[pai]); //troca nó com pai
      subir(H, pai);
    }
}
```





Lembrete:

A função `descer(lista H, int i)`:

- *Considera que as sub-árvores esquerda e direita são heaps;*
- *Transforma a árvore enraizada em `i` em uma heap;*

A função `subir(lista H, int i)`:

- *Considera que `H[1,...i-1]` é heap;*
- *Transforma `H[1,...i1]` em uma heap;*



Ideia 1: Olhar os elementos um a um e considerar uma nova inserção.

Ideia 2: Uma vez as folhas são heaps, arranjar os nós que não são folha

Pseudocódigo

```
ideia1{
    para (i = 1 até n)
      subir(H,i);
}//O(n log n)
```

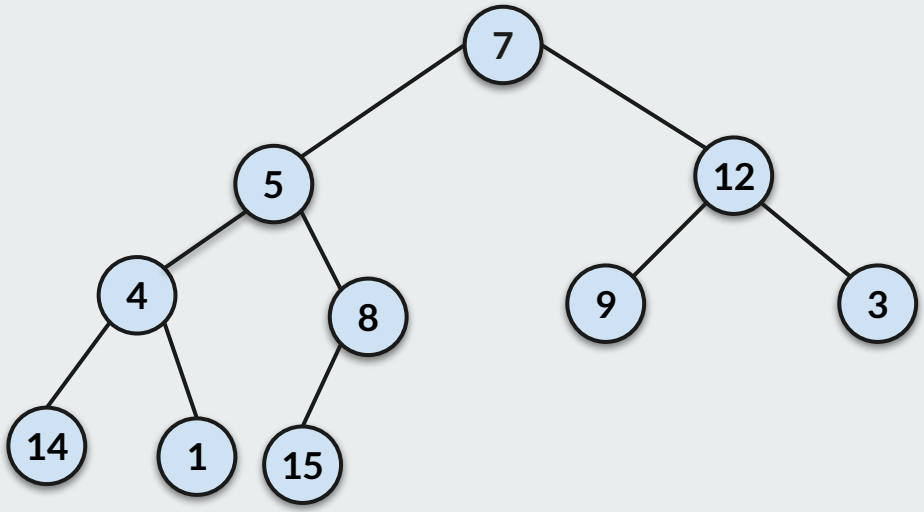


Pseudocódigo

```
ideia2{
   para (i =⌊n/2⌋ até 1)
     descer(H,i);
} //O(n)
```

Construir Heap

1	2	3	4	5	6	7	8	9	10
7	5	12	4	8	9	3	14	1	15



A função `descer(lista H, int i)`:

- *Considera que as sub-árvores esquerda e direita são heaps;*
- *Transforma a árvore enraizada em `i` em uma heap;*

A função `subir(lista H, int i)`:

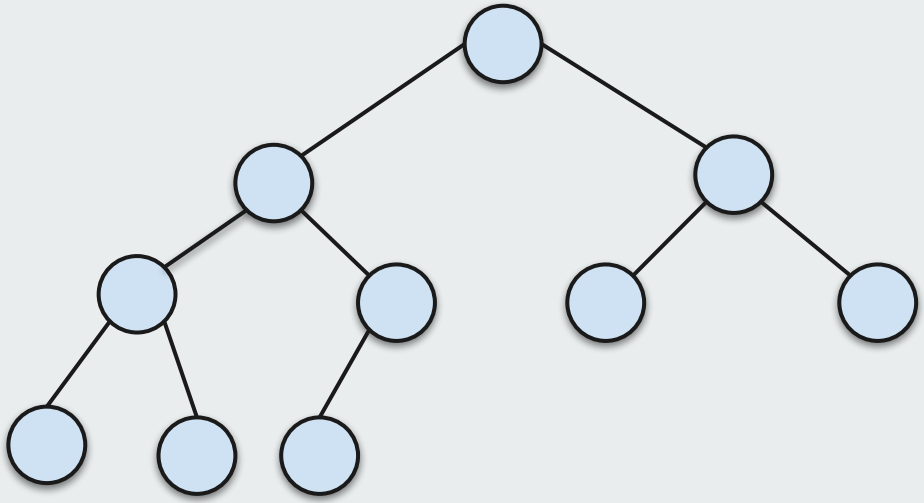
- *Considera que `H[1,...i-1]` é heap;*
- *Transforma `H[1,...i1]` em uma heap;*

Construir Heap

1	2	3	4	5	6	7	8	9	10
7	5	12	4	8	9	3	14	1	15

A função `descer(lista H, int i)`:

- *Considera que as sub-árvores esquerda e direita são heaps;*

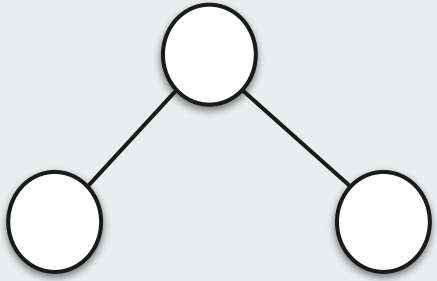


Pseudocódigo

```
ideia2{
  para (i =⌊n/2⌋ até 1)
    descer(H,i);
}
```

Pseudocódigo

```
construir(lista H){
  para (i =⌊H.tamanho/2⌋ até 1)//decrescente
    descer(H,i);
}
```





HEAPSORT

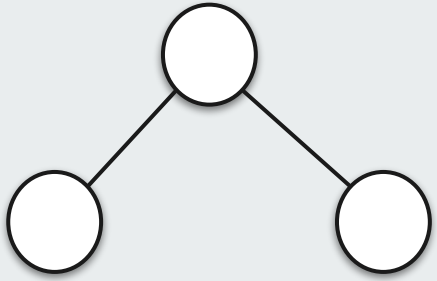
1. Organizar os elementos como uma *heap* (procedimento **construir**).

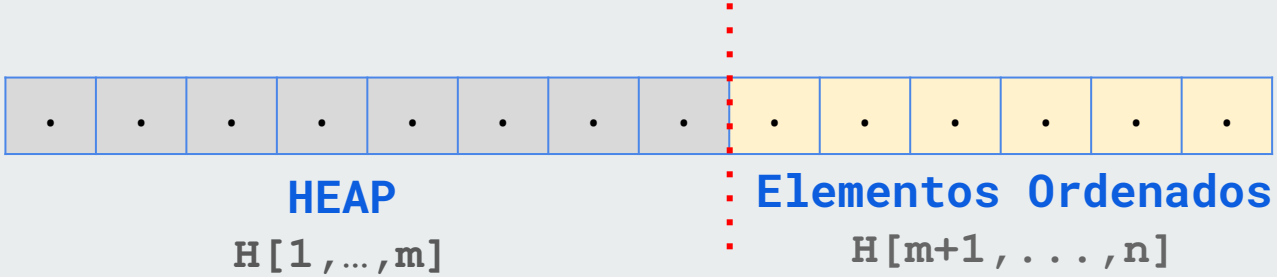
2. Uma vez que o elemento de maior prioridade se encontra na primeira posição, `H[1]`, basta removê-lo (colocando na última posição).

3. Repetir o processo para os `(n – 1)` elementos da *heap*. Isto deve ser feito até a *heap* ter tamanho 2.

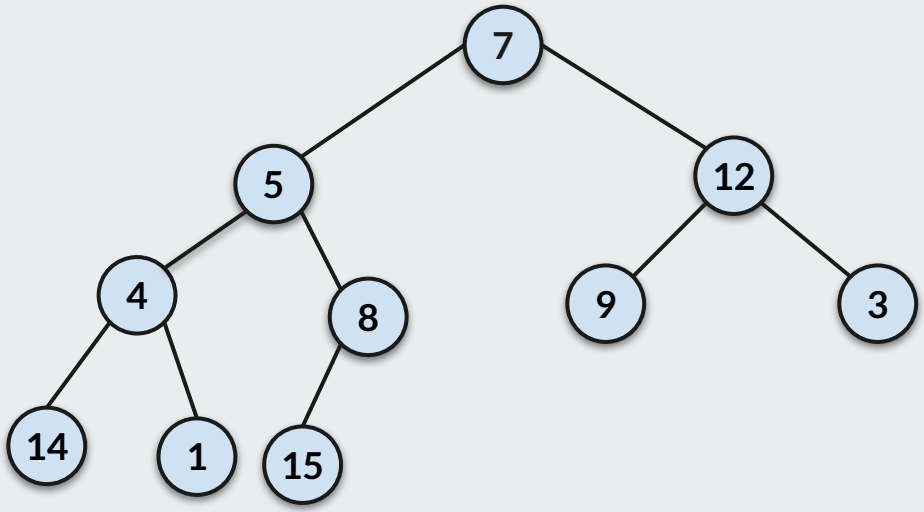
Pseudocódigo

```
heapsort(lista H){
   construir(H);     //O(n)
   para (i=H.tamanho até 2){  //O(n)
      troca(H[1],H[i]);
      H.tamanho = H.tamanho-1;
      descer(H, 1); //O(log n)
   }
}   //O(n log n)
```





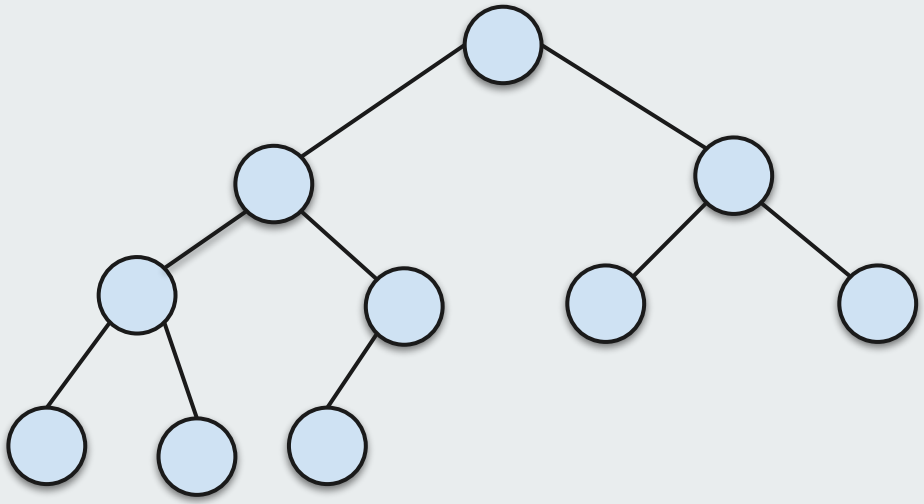
1	2	3	4	5	6	7	8	9	10
7	5	12	4	8	9	3	14	1	15



Pseudocódigo

```
heapsort(lista H){
  construir(H);
  para (i=H.tamanho até 2){
    troca(H[1],H[i]);
    H.tamanho = H.tamanho-1;
    descer(H, 1);
  }
}
```

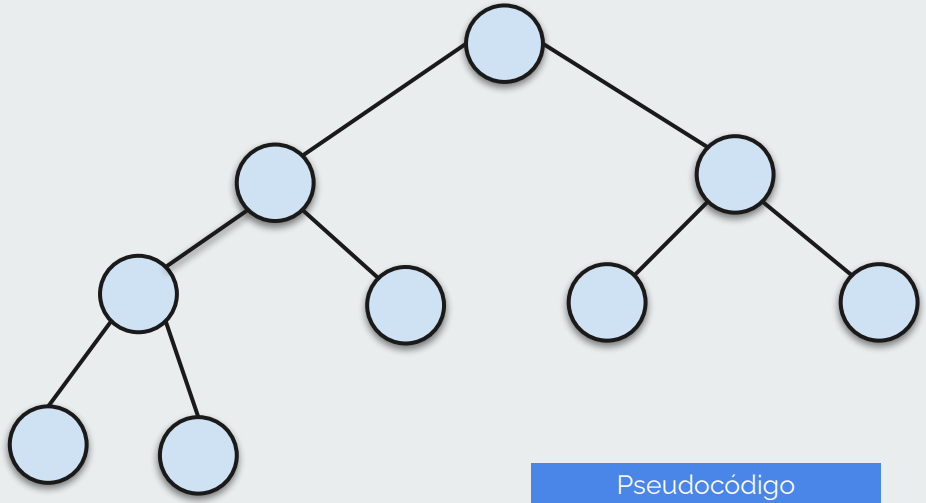
	1	2	3	4	5	6	7	8	9	10
H inicial	7	5	12	4	8	9	3	14	1	15
H heap										
H ordenado										



Pseudocódigo

```
heapsort(lista H){
   construir(H);
   para (i=H.tamanho até 2){
      troca(H[1],H[i]);
      H.tamanho = H.tamanho-1;
      descer(H, 1);
   }
}
```

1	2	3	4	5	6	7	8	9
6	1	8	7	3	9	5	2	4



Pseudocódigo

```
heapsort(lista H){
   construir(H);
   para (i=H.tamanho até 2){
      troca(H[1],H[i]);
      H.tamanho = H.tamanho-1;
      descer(H, 1);
   }
}
```

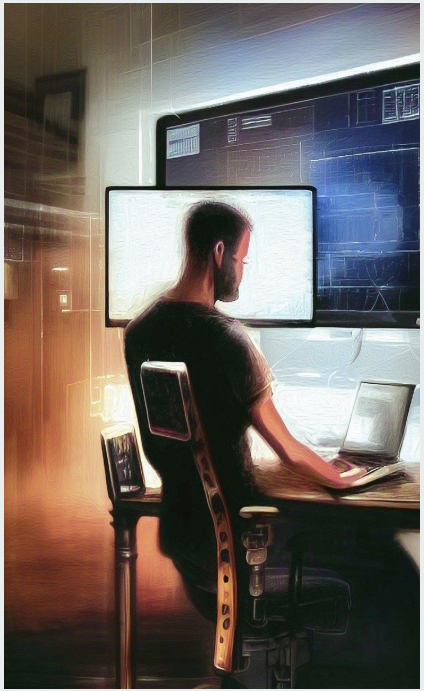
Vamos programar?



1. Implemente o procedimento de criação da heap a partir de um vetor qualquer;

2. Implemente o Heapsort.





Dúvidas?!

